

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Conexión

Ejercicios Prácticos Avanzados

Soluciones de Referencia

Consolidación y Retroalimentación

Conclusiones

Apéndice: Ver también

Introducción y Conexión

Conceptos fundamentales que aprendimos

- Ajuste con **curve_fit**: Funciones arbitrarias y no lineales
- Matriz de covarianza: Extracción automática de incertidumbres
- Ejemplo físico: Relación masa-luminosidad (log-log vs no lineal)

Objetivo de hoy (P2)

Aplicar técnicas avanzadas: **interpolación** de datos faltantes y **selección de modelos** con criterios estadísticos.

Técnicas avanzadas para análisis de datos reales:

1. Interpolación (Ejercicio 5):

- Reconstruir datos con observaciones incompletas
- Comparar métodos: lineal vs cúbico
- Aplicación: Curvas de luz con gaps

2. Selección de modelos (Ejercicio 6):

- Usar AIC para elegir entre modelos competidores
- Comparar distribuciones: Normal vs Lognormal
- Interpretación física de asimetrías

¡Estas técnicas son esenciales en astronomía observacional!

Ejercicios Prácticos Avanzados



Curva de luz con datos faltantes

Enunciado

- Estrella variable: $F(t) = 1.0 + 0.1 \sin(2\pi t/P)$ con $P = 2.5$ días
- Genere 60 puntos en $t \in [0, 10]$ días
- **Simule observación:** Elimine 20 puntos al azar (nubes, mal clima)
- **Tareas:**
 1. Cree interpoladores lineal y cúbico con `interp1d`
 2. Evalúe ambos en grilla fina (200 puntos)
 3. Grafique: curva original + observaciones + interpolaciones
 4. Compare visualmente: ¿cuál sigue mejor la señal?
 5. **Extra:** Calcule error RMS entre interpolación y original

Conceptos: Interpolación, comparación lineal vs cúbica

Física: Reconstruir curvas de luz con observaciones incompletas



Selección de modelo: Lognormal vs Normal

Enunciado

- Datos de masas de asteroides (típicamente asimétricos hacia valores altos)
- Genere 1500 valores con distribución Lognormal:
 - $\mu = 0, \sigma = 0.6$
- **Pregunta:** ¿Qué modelo describe mejor los datos: Lognormal o Normal?
- **Tareas:**
 1. Ajuste ambas distribuciones con `scipy.stats`
 2. Calcule AIC para cada modelo: $AIC = 2k - 2 \ln(L)$
 3. Grafique histograma + PDFs de ambos modelos
 4. Concluya: ¿cuál modelo es mejor? (menor AIC)
 5. Interprete: ¿por qué la Normal falla?

Conceptos: AIC, selección de modelos, log-verosimilitud

Aplicación: Elegir distribución correcta para datos astronómicos

Soluciones de Referencia

Dos técnicas avanzadas

- Ejercicio 5: Interpolación con `scipy.interpolate`
- Ejercicio 6: Selección de modelos con AIC

Estas soluciones muestran técnicas que se usan en investigación real.

Soluciones de Referencia $\ni$ Solución 5:

Interpolación de curva de luz



```
1 from scipy.interpolate import interp1d
2 import random, math
3 import matplotlib.pyplot as plt
4
5 # Generar curva de luz completa
6 P = 2.5 # periodo
7 t_full = [i*0.05 for i in range(201)] # 0 a 10 en 201 puntos
8 F_full = [1.0 + 0.1*math.sin(2*math.pi*t/P) for t in t_full]
9
10 # Eliminar 20 puntos al azar
11 random.seed(42)
12 indices = random.sample(range(len(t_full)), len(t_full) - 20)
13 indices.sort()
14 t_obs = [t_full[i] for i in indices]
15 F_obs = [F_full[i] for i in indices]
16
17 # Crear interpoladores
18 f_lin = interp1d(t_obs, F_obs, kind='linear')
19 f_cub = interp1d(t_obs, F_obs, kind='cubic')
20
21 # Evaluar en grilla fina
22 t_interp = [i*0.025 for i in range(401)] # más fino para suavidad
23 F_lin = [f_lin(t) for t in t_interp]
24 F_cub = [f_cub(t) for t in t_interp]
```

Soluciones de Referencia $\ni$ Solución 5: Interpolación (gráficos)



```
23 # Graficar comparación
24 plt.figure(figsize=(10, 6))
25 plt.plot(t_full, F_full, 'k-', alpha=0.3, linewidth=2, label='Original')
26 plt.scatter(t_obs, F_obs, s=20, c='black', zorder=5, label='Observaciones')
27 plt.plot(t_interp, F_lin, 'b--', linewidth=1.5, label='Lineal')
28 plt.plot(t_interp, F_cub, 'r-', linewidth=1.5, label='Cúbica')
29 plt.xlabel('Tiempo')
30 plt.ylabel('Flujo F(t)')
31 plt.title('Interpolación de Curva de Luz')
32 plt.legend()
33 plt.grid(True, alpha=0.3)
34 plt.show()
35
36 # Comentario: La interpolación cúbica sigue mejor la forma suave
37 # de la señal original, mientras que la lineal produce segmentos
38 # rectos entre puntos. Ambas son exactas en los puntos observados.
```

Nota: cúbica captura mejor la variación suave; lineal es más robusta al ruido.

Interpolación lineal vs cúbica

- **Lineal:** Conecta puntos con rectas $\rightarrow$ simple, robusta, no suave
- **Cúbica:** Usa splines cúbicos $\rightarrow$ suave, captura curvaturas, puede oscilar

¿Cuándo usar cada una?

- **Lineal:** Datos ruidosos, pocos puntos, necesita robustez
- **Cúbica:** Señal suave subyacente, alta densidad de puntos
- **Nunca extrapolar:** Interpolación solo válida dentro del rango de datos

Error RMS:
$$\text{RMS} = \sqrt{\frac{1}{N} \sum (F_{\text{interp}} - F_{\text{real}})^2}$$

Un RMS bajo indica que la interpolación reconstruye bien la señal original.

Aplicación en astronomía: Rellenar huecos en curvas de luz, tránsitos de exoplanetas, variabilidad estelar.



Lognormal vs Normal

```
1 from scipy import stats
2 import random
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 # Generar datos lognormales
7 random.seed(42)
8 n = 1500
9 mu, sigma = 0.0, 0.6
10 x = [random.lognormvariate(mu, sigma) for _ in range(n)]
11
12 # Ajustar distribución Lognormal
13 s_log, _, scale_log = stats.lognorm.fit(x, floc=0)
14 ll_log = sum(stats.lognorm.logpdf(xi, s_log, 0, scale_log) for xi in x)
15 k_log = 2 # parámetros: s y scale
16 AIC_log = 2*k_log - 2*ll_log
17
18 # Ajustar distribución Normal
19 mu_norm, sig_norm = stats.norm.fit(x)
20 ll_norm = sum(stats.norm.logpdf(xi, mu_norm, sig_norm) for xi in x)
21 k_norm = 2 # parámetros: mu y sigma
22 AIC_norm = 2*k_norm - 2*ll_norm
23
24 print(f"Lognormal: AIC = {AIC_log:.1f}")
25 print(f"Normal: AIC = {AIC_norm:.1f}")
26 print(f"Mejor modelo: {'Lognormal' if AIC_log < AIC_norm else 'Normal'}")
```

Soluciones de Referencia $\ni$ Solución 6:

Lognormal vs Normal (gráficos)



```
24 # Graficar histograma y PDFs ajustadas
25 plt.figure(figsize=(10, 6))
26 plt.hist(x, bins=40, density=True, alpha=0.5, edgecolor='black',
27          label='Histograma')
28
29 # Generar curvas de densidad
30 x_plot = np.linspace(0, max(x), 500)
31 pdf_log = stats.lognorm.pdf(x_plot, s_log, 0, scale_log)
32 pdf_norm = stats.norm.pdf(x_plot, mu_norm, sig_norm)
33
34 plt.plot(x_plot, pdf_log, 'r-', linewidth=2,
35          label=f'Lognormal (AIC={AIC_log:.1f})')
36 plt.plot(x_plot, pdf_norm, 'b--', linewidth=2,
37          label=f'Normal (AIC={AIC_norm:.1f})')
38
39 plt.xlabel('Valor')
40 plt.ylabel('Densidad de Probabilidad')
41 plt.title('Comparación de Ajustes: Lognormal vs Normal')
42 plt.legend()
43 plt.grid(True, alpha=0.3)
44 plt.show()
```

Conclusión: Lognormal tiene menor AIC y captura mejor la asimetría positiva.

Criterio de Información de Akaike (AIC)

- $AIC = 2k - 2 \ln(L)$ donde k = número de parámetros, L = verosimilitud
- Penaliza modelos complejos (muchos parámetros)
- **Regla:** El modelo con **menor AIC** es preferible
- Diferencia $\Delta AIC > 10$: Evidencia fuerte a favor del mejor modelo

¿Por qué la Normal falla?

- Datos Lognormales son **asimétricos** (cola larga hacia valores altos)
- La Normal es **simétrica** $\rightarrow$ no puede capturar la asimetría
- Forzar una Normal produce residuos sistemáticos

Interpretación física: Muchas magnitudes astrofísicas son lognormales (masas, luminosidades, tiempos) debido a procesos multiplicativos. Usar Normal lleva a conclusiones erróneas.

Conclusión: Siempre visualizar datos antes de elegir modelo. El AIC cuantifica lo que el ojo ve.

Consolidación y Retroalimentación

Reflexión sobre técnicas avanzadas

Interpolación (Ejercicio 5):

- ¿Cuándo preferirían interpolación lineal vs cúbica?
- ¿Notaron oscilaciones en la cúbica en algunos casos?

Selección de modelos (Ejercicio 6):

- ¿El AIC les ayudó a decidir entre modelos objetivamente?
- ¿Lograron visualizar por qué la Normal no captura la asimetría?

Objetivo

Desarrollar intuición para elegir métodos apropiados según las características de los datos.

Errores frecuentes

- **Extrapolar interpolaciones:** Solo válidas dentro del rango de datos
- **Elegir modelo por "se ve bien":** El AIC cuantifica objetivamente
- **Ignorar asimetrías:** Usar Normal para datos claramente asimétricos
- **Sobre-confiar en interpolación cúbica:** Puede oscilar con ruido
- **No validar con datos reales:** Siempre comparar con ground truth cuando sea posible

Consejo

Workflow recomendado: Visualizar $\rightarrow$ Probar múltiples métodos $\rightarrow$ Comparar métricas $\rightarrow$ Elegir con criterio

Conclusiones

Técnicas avanzadas dominadas hoy

Interpolación:

- Uso de `scipy.interpolate.interp1d`
- Comparación lineal vs cúbica
- Evaluación con RMS

Selección de modelos:

- Criterio AIC para comparación objetiva
- Ajuste de distribuciones con `scipy.stats`
- Interpretación física de asimetrías

Habilidad adquirida

Reconstruir datos incompletos y elegir modelos estadísticos apropiados para fenómenos físicos reales.

Librerías esenciales

- `scipy.interpolate.interp1d`: Interpolación 1D (lineal, cúbica, etc.)
- `scipy.stats`: Distribuciones, ajuste y pruebas estadísticas
- `numpy`: Operaciones vectoriales y cálculo de métricas
- `matplotlib`: Visualización comparativa

Métricas y criterios

- **RMS**: Error de reconstrucción en interpolación
- **AIC**: Selección de modelos (menor es mejor)
- **Visualización**: Histogramas + PDFs para comparar distribuciones

- **Próxima semana:** Integración de todas las técnicas en proyectos completos
- **Recomendación de estudio:**
 - Explorar otros tipos de interpolación (nearest, quadratic)
 - Leer sobre otros criterios de selección (BIC, validación cruzada)
- **Práctica sugerida:**
 - Aplicar interpolación a datasets reales (Kaggle, astroML)
 - Comparar AIC vs BIC en problemas propios

¡Excelente trabajo en estas dos semanas!

Apéndice: Ver también

```
1 from scipy.interpolate import interp1d
2
3 # Opciones de kind:
4 # 'linear', 'nearest', 'zero', 'slinear', 'quadratic', 'cubic'
5
6 f_nearest = interp1d(x, y, kind='nearest')
7 f_quadratic = interp1d(x, y, kind='quadratic')
8
9 # Para datos 2D: ver scipy.interpolate.interp2d
10 # Para datos irregulares: ver scipy.interpolate.griddata
```

```
1 # Similar a AIC pero penaliza más la complejidad
2 BIC = k * np.log(n) - 2 * log_likelihood
3
4 # Comparar con AIC:
5 # BIC penaliza más modelos complejos (útil con muchos datos)
6 # AIC es mejor para predicción
7 # Ambos: menor es mejor
```

```
1 from scipy import stats
2
3 # Comparar datos con distribución teórica
4 D, p_value = stats.kstest(x, 'norm', args=(mu, sigma))
5
6 # D: estadístico KS (distancia máxima entre CDFs)
7 # p_value > 0.05: no rechazamos la hipótesis (datos ~ Normal)
8 # p_value < 0.05: rechazamos (datos NO son Normal)
9
10 print(f"KS: D={D:.3f}, p={p_value:.3f}")
```

Nota: Complementa al AIC para validación de modelos.